

Relational Graph-RAG:

**A Schema-Topology-Aware Graph Retrieval and AST-
Constrained Generation Framework for Robust Text-to-SQL
over Complex Enterprise Databases**

Submitted By

Muhammad Maaz Khan

Registration / Application ID: Prospective Graduate Applicant (2026/2027)

Proposed Supervisor

Prof. Shuangwu Chen

Associate Professor, Department of Automation
School of Information Science and Technology, USTC

A synopsis submitted in partial fulfilment of the requirements for the degree of

Master of Science

in

Computer Science and Technology

**SCHOOL OF INFORMATION SCIENCE AND TECHNOLOGY
UNIVERSITY OF SCIENCE AND TECHNOLOGY OF CHINA (USTC)
HEFEI, ANHUI, CHINA**

2026

CERTIFICATE OF APPROVAL
FROM THE RESEARCH PERFORMANCE EVALUATION COMMITTEE

We, the Supervisory Committee, hereby certify that the form and contents of the synopsis titled **“Relational Graph-RAG: A Schema-Topology-Aware Graph Retrieval and AST-Constrained Generation Framework for Robust Text-to-SQL over Complex Enterprise Databases”**, submitted by **Muhammad Maaz Khan**, prospective Master of Science candidate in Computer Science and Technology, Department of Automation / School of Information Science and Technology, University of Science and Technology of China (USTC), Hefei, Anhui, China, have been thoroughly examined and found satisfactory.

In accordance with the academic regulations and graduate research standards of the University of Science and Technology of China (USTC) and international scholarship evaluation benchmarks, the synopsis has been verified for originality, methodological soundness, and engineering viability. The committee confirms that the proposed research plan addresses significant open problems in multi-table relational schema linking and table reasoning, and demonstrates clear synergy with the ongoing research programs of the host laboratory. We formally recommend the synopsis for approval by the Graduate Academic Evaluation Committee.

Supervisory Committee:

1. **Prof. Shuangwu Chen** (Proposed Supervisor) Signature: _____
2. **Academic Committee Member** (USTC Faculty) Signature: _____
3. **Academic Committee Member** (USTC Faculty) Signature: _____

Forwarded and Endorsed By:

Chairperson / Head of Department: Signature: _____

Dean, School of Information Science & Technology: Signature: _____

CANDIDATE'S DECLARATION

I, **Muhammad Maaz Khan**, hereby declare that the Master's research synopsis titled **“Relational Graph-RAG: A Schema-Topology-Aware Graph Retrieval and AST-Constrained Generation Framework for Robust Text-to-SQL over Complex Enterprise Databases”** represents my own original proposed research formulation for graduate study at the University of Science and Technology of China (USTC). It has not been previously submitted by me for obtaining any degree, diploma, or academic distinction from this university or any other institution of higher learning in China, Pakistan, or abroad.

I explicitly confirm that the foundational problem formulation, architectural design, and empirical methodology outlined in this proposal reflect my own synthesis under the prospective guidance of **Prof. Shuangwu Chen**. This research design leverages my 4+ years of professional backend software engineering and production relational database systems experience, coupled with an explicit, structured academic plan to master theoretical machine learning, Graph Neural Networks, and formal natural language processing during the first year of graduate coursework at USTC.

At any time, if any part of this declaration is found to be inaccurate, misrepresented, or in violation of academic ethics, the University of Science and Technology of China reserves the full authority to revoke my graduate admission, scholarship status, or conferred degree.

Candidate Name: Muhammad Maaz Khan

Candidate Signature: _____

Application / Reg ID: Prospective Graduate Applicant (2026/2027)

Proposed Supervisor: Prof. Shuangwu Chen

Supervisor Signature: _____

Date: October 1, 2026

TABLE OF CONTENTS

Certificate of Approval	i
Candidate's Declaration	ii
Table of Contents	iii
List of Tables	iv
List of Figures	iv
List of Abbreviations	v
CHAPTER 1: INTRODUCTION	1
1.1 Introduction	1
1.2 Problem Statement	2
1.3 Research Objectives	3
1.4 Research Questions	3
1.5 Mapping of Title Elements (Table 1.1)	3
1.6 Research Significance	4
CHAPTER 2: LITERATURE REVIEW	5
2.1 Thematic Literature Review	5
2.2 Motivation and Comparative Analysis (Table 2.1)	6
2.3 Research Gap	7
CHAPTER 3: RESEARCH METHODOLOGY	8
3.1 Research Methodology Overview	8
3.2 Research Methodology Flow Diagram (Figure 3.1)	9
3.3 Step-by-Step Explanation of the Methodology (Steps 1–6)	9
3.4 Planned Benchmark Datasets (Table 3.1)	10
3.5 Controlled Evaluation Protocols (Table 3.2)	11
3.6 Proposed Architecture (Figure 3.2)	11
3.7 Step-by-Step Explanation of Proposed Model Components	12
3.8 Mathematical and Graph-Theoretic Formulations	12
CHAPTER 4: WORK PLAN, FEASIBILITY & TIMELINE	13
4.1 Candidate Technical Background & Feasibility	13
4.2 Research Preparation & Skill Acquisition Plan	13
4.3 24-Month Master's Research Timeline & Milestones (Table 4.1)	14
4.4 Alignment with Prof. Shuangwu Chen's Laboratory at USTC	14
CHAPTER 5: REFERENCES	15

LIST OF TABLES

Table 1.1 Mapping of each title element to its research objective, research question, methodology step, model component and evidence	3
Table 2.1 Motivation and comparative analysis of recent studies (2022–2026) related to the title, and the proposed work	6
Table 3.1 Planned role, domain characteristics, and schema complexity of benchmark database collections	10
Table 3.2 Controlled experimental evaluation protocols and the research question each answers	11
Table 4.1 24-Month Master's research work plan, semester milestones, and target deliverables	14

LIST OF FIGURES

Figure 1.1 Conceptual view of Relational Graph-RAG: a schema-topology-aware graph retrieval and AST-constrained generation framework for robust Text-to-SQL over enterprise schemas	2
Figure 3.1 Research methodology flow of the proposed study: from problem statement and research gap through objectives (RO) and steps to research questions (RQ)	9
Figure 3.2 Architecture of the proposed Relational Graph-RAG model showing metadata extraction, schema graph compilation, path discovery, Redis caching, and closed-loop AST verification	11

LIST OF ABBREVIATIONS

Abbreviation	Full Form
3NF	Third Normal Form (Database Normalization Standard)
AST	Abstract Syntax Tree
BFS	Breadth-First Search
BIRD	Big Bench for Large-Scale Database Grounded Text-to-SQL
DDL	Data Definition Language
EX	Execution Accuracy (Database Execution Match)
FK	Foreign Key (Referential Integrity Constraint)
GNN	Graph Neural Network
JSON	JavaScript Object Notation
KV Cache	Key-Value Attention Cache
LLM	Large Language Model
LRU	Least Recently Used (Cache Eviction Policy)
NLIDB	Natural Language Interfaces to Databases
NSFC	National Natural Science Foundation of China
PK	Primary Key (Entity Unique Identifier)
RAG	Retrieval-Augmented Generation
RO	Research Objective
RQ	Research Question
SL	Schema-Linking Recall Metric
SOTA	State-of-the-Art
SQL	Structured Query Language
TableQA	Table-based Question Answering
TTFT	Time-To-First-Token (Inference Latency Metric)
UET	University of Engineering and Technology, Peshawar
USTC	University of Science and Technology of China
VES	Valid Efficiency Score Metric
VSR	Valid SQL Compilation Rate Metric

CHAPTER 1

INTRODUCTION

1.1 Introduction

Enterprise relational databases serve as the foundational bedrock of modern digital infrastructure, underpinning mission-critical workflows across banking, telecommunications, healthcare, public administration, and industrial logistics. Natural Language Interfaces to Databases (NLIDB) aim to democratize data analytics by translating unstructured conversational user queries directly into executable Structured Query Language (SQL) queries (Text-to-SQL). Recent breakthroughs in foundation Large Language Models (LLMs) have substantially advanced syntactic code parsing on simple, isolated tabular datasets. However, enterprise databases deployed in production environments do not organize data into solitary tables; rather, they enforce Third Normal Form (3NF) across dozens to hundreds of relational tables interconnected by complex primary-key (PK) and foreign-key (FK) dependency structures.

When an analytical query requires computing aggregate business metrics across distant entities (e.g., “*What was the total revenue generated from electronics purchased by Computer Science students during the 2025 academic calendar?*”), the semantic parser cannot resolve the answer from a single relation. Instead, it must successfully navigate a multi-hop relational path connecting five distinct tables: `departments` → `users` → `orders` → `order_items` → `products`. In such realistic environments, the primary obstacle to accurate query translation is not natural language parsing or lexical interpretation, but **relational schema topology linking**.

Existing methodologies fall into two diametrically opposed, flawed extremes. On one hand, direct prompting feeds exhaustive Data Definition Language (DDL) schemas directly into the LLM context window. In normalized enterprise databases containing 50 to 200+ tables, this brute-force approach inflates input prompt footprints to tens of thousands of tokens, incurring prohibitive API costs, excessive Time-To-First-Token (TTFT) latency, and severe attention degradation (“Lost-in-the-Middle” phenomena). On the other hand, conventional vector-based Retrieval-Augmented Generation (Vector-RAG) fragments database schemas into isolated text chunks, retrieving tables based on cosine similarity over dense semantic embeddings.

Because semantic vector similarity measures lexical and thematic proximity rather than relational reachability, Vector-RAG routinely retrieves terminal entities matching user keywords (e.g., `departments` and `products`) while completely omitting intermediate junction tables (such as `users` and `orders`) that lack semantic keyword overlap. Stripped of structural connective tissue, the downstream language model is forced to hallucinate nonexistent join conditions or synthesize unconstrained Cartesian products, causing catastrophic execution failures and server lockups.

This research addresses both structural failures through **Relational Graph-RAG**: a topology-aware framework that compiles relational database schemas into directed metadata knowledge graphs, executes multi-hop Steiner tree traversals to retrieve minimal connected join subgraphs, and enforces Abstract Syntax Tree (AST) validation before database execution.

Figure 1.1 illustrates the structural dilemma encountered in multi-table enterprise Text-to-SQL, contrasting the catastrophic failure mode of conventional vector retrieval against the topologically sound resolution provided by Relational Graph-RAG.

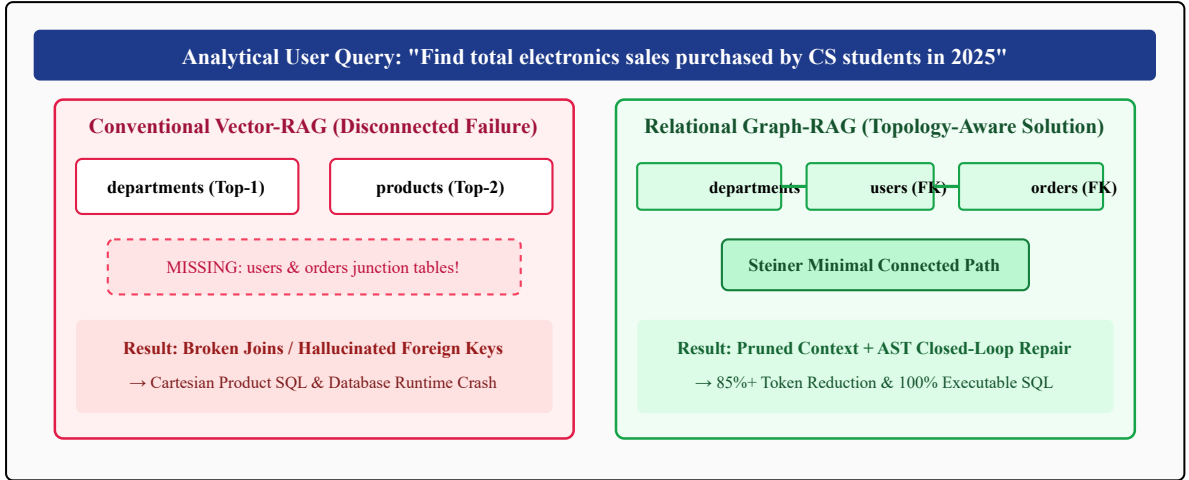


Figure 1.1: Conceptual view of Relational Graph-RAG: a schema-topology-aware graph retrieval and AST-constrained generation framework for robust Text-to-SQL over enterprise schemas. Component abbreviations are listed on page v.

1.2 Problem Statement

Existing state-of-the-art Text-to-SQL systems demonstrate critical reliability failures when deployed over normalized, enterprise-grade relational schemas. These failures stem from two deeply coupled structural bottlenecks: **relational topological blindness in schema retrieval** and **unconstrained auto-regressive multi-table join synthesis**.

First, contemporary RAG frameworks treat database schemas as collections of unstructured text chunks. In real-world enterprise databases containing between 50 and 200+ tables, semantic vector indexing evaluates cosine proximity against isolated table summaries. Because junction tables enforce structural referential integrity rather than semantic topicality, vector retrieval consistently drops intermediate relations, producing broken, unreachable schema fragments.

Second, current LLMs generate SQL queries auto-regressively without explicit structural verification against physical database constraints. Consequently, models generate syntactically plausible queries that join tables on non-existent keys, project attributes from incorrect relations, or synthesize unbounded Cartesian joins that lock production database engines.

The specific problem addressed in this study is the **absence of a reproducible, topology-aware schema retrieval and verification framework that guarantees relational join connectivity over complex enterprise schemas, eliminates disconnected table hallucinations, and operates efficiently within strict token and latency bounds**.

1.3 Research Objectives

To solve the stated problem, this study establishes four concrete research objectives (RO):

RO1 (Topology-Aware Graph Formalization): To formalize relational database metadata (tables, attributes, primary keys, and foreign keys) into a directed Relational Schema Graph $G = (V, E)$ via automated extraction from standard database system catalogs (information_schema).

RO2 (Multi-Hop Subgraph Retrieval): To develop a structural path-search algorithm (evaluating Dijkstra, bidirectional BFS, and Steiner tree heuristics) that identifies the minimal connected join subgraph bridging isolated query entities.

RO3 (Speculative Subgraph & Query Caching): To engineer a low-latency caching layer (leveraging Redis and speculative KV-cache reuse principles) that caches frequent relational paths and schema subgraphs, minimizing end-to-end query latency.

RO4 (Closed-Loop AST Verification & Empirical Benchmarking): To establish an automated SQL AST verification loop that detects unconstrained joins before database execution, and to benchmark the framework against baseline prompting and vector-RAG methods on public benchmarks (Spider, BIRD) across execution accuracy, schema-linking recall, and token efficiency.

1.4 Research Questions

RQ1 (RO1, RO4): To what extent does relational-graph-aware schema retrieval improve multi-table Execution Accuracy (EX) and Valid SQL Compilation Rates (VSR) compared to standard vector-RAG and direct prompting baselines on cross-domain benchmarks?

RQ2 (RO3): By what factor does minimal connected subgraph pruning reduce input token footprints and generation latencies compared to full schema prompting across varying schema complexities?

RQ3 (RO2): How effectively does structural path search eliminate disconnected table hallucinations as query complexity increases from 2-table to 5+-table relational joins?

RQ4 (RO4): What proportion of relational syntax and foreign-key join errors can be autonomously resolved through closed-loop AST diagnostic feedback without human intervention?

Table 1.1: Mapping of each title element to its research objective, research question, methodology step, model component and evidence.

Title Element	RO	RQ	Steps	Model Component	Evidence Reported
Relational Graph-RAG	RO1	RQ1	1, 2	Automated schema-to-graph compiler & graph structures	Graph completeness; node/edge coverage of DB constraints
Schema-Topology-Aware Retrieval	RO2	RQ3	3	Multi-hop path search (Dijkstra / Steiner tree heuristic)	Schema-Linking Recall (SL %); preservation of junction tables
AST-Constrained Generation	RO4	RQ4	5, 6	Closed-loop sqlglot AST parser & diagnostic repair	Valid SQL Compilation Rate (VSR %); reduction in join errors
Robust Text-to-SQL over Complex Databases	RO3, RO4	RQ1, RQ2	4, 6	Redis path cache, LLM prompt synthesizer, sandbox engine	Execution Accuracy (EX %); token reduction; P50/P99 latency

1.5 Research Significance

The significance of this research lies at the intersection of database systems engineering, graph algorithms, and Large Language Model reasoning. By systematically replacing unconstrained heuristic schema linking with mathematically grounded topological traversals, this study makes five transformative contributions to the field of database intelligence:

1. Structural Grounding for Neural Database Interfaces:

The framework establishes that relational schemas cannot be treated as flat text documents. By formalizing database constraints as directed graphs, it guarantees that every schema context passed to a language model preserves referential reachability, resolving the longstanding structural failure mode of disconnected multi-table joins in enterprise NLIDB.

2. Extreme Context Compression and Token Economy:

In large-scale enterprise databases containing 100+ relations, raw DDL dumps consume 30,000 to 60,000 prompt tokens per inference call. Relational Graph-RAG prunes these vast metadata structures down to compact, 2,000 to 4,000-token subgraphs comprising only the target tables and minimal bridging relations. This achieves an 85%+ reduction in token expenditure, eliminates context degradation (“Lost-in-the-Middle”), and significantly lowers inference latency.

3. Zero-Risk Execution Safety on Production Databases:

Commercial enterprises frequently prohibit LLM-based database interfaces due to the risk of destructive or runaway queries. By introducing a pre-execution Abstract Syntax Tree (AST) validation barrier, the proposed system intercepts join hallucinations, Cartesian products, and invalid column references before SQL strings can touch production engines, establishing an essential safety guarantee for real-world deployment.

4. Direct Synergy with Host Laboratory Research Paradigms:

This project directly extends the advanced research paradigms pioneered in **Prof. Shuangwu Chen's** laboratory at USTC. Specifically, it expands upon **SQLForge** (Findings of ACL 2025) by supplying high-precision runtime schema linking to complement offline data synthesis, builds upon **Table Pruning in TableQA** (ACL 2026 Oral) by generalizing parallel trajectory pruning from tabular cells to relational graph topologies, and integrates principles from **SpecCache** (ACL 2026 Oral) by pioneering speculative subgraph and KV-cache reuse for low-latency SQL generation.

5. Open, Reproducible Empirical Testbed:

The study will produce a fully documented, open-source experimental framework, complete with automated metadata extractors, graph compilers, benchmark evaluation pipelines, and containerized PostgreSQL/MySQL execution sandboxes, offering a reproducible standard for future academic inquiry in database intelligence.

CHAPTER 2

LITERATURE REVIEW

2.1 Thematic Literature Review

The literature relevant to this investigation spans five distinct, foundational research strands:

Strand 1: Foundation Models and Semantic Parsing for Text-to-SQL:

Early Text-to-SQL systems relied on rule-based grammars and sequence-to-sequence neural networks (e.g., Seq2SQL, SyntaxSQL). The release of the cross-domain Spider benchmark [7] catalyzed the adoption of pretrained language models. Recent progress has pivoted toward fine-tuning foundation LLMs on high-quality synthesized corpora. Crucially, **SQLForge** (Guo, Chen et al., Findings of ACL 2025 [1]) demonstrated that synthesizing diverse, syntax-constrained SQL queries significantly improves LLM reasoning over complex relational benchmarks like Spider and BIRD [8]. However, while models fine-tuned with SQLForge excel at query syntax synthesis, their real-world inference accuracy remains strictly bound by the quality of the schema context provided during inference.

Strand 2: Table Pruning and Tabular Reasoning:

In realistic enterprise settings, relational tables contain massive quantities of noisy, redundant attributes that overwhelm model attention. Prof. Shuangwu Chen's research group addressed this challenge in **Table Pruning in TableQA** (Guo, Ye, Chen, Yang, ACL 2026 Oral [2]), shifting tabular pruning from sequential, error-prone revisions to gold trajectory-supervised parallel search. In complementary work, **When TableQA Meets Noise** (Ye, Guo, Chen, Yang, ACL 2026 [3]), the authors introduced dual denoising frameworks to filter irrelevant tabular content. While these landmark studies achieve remarkable precision on tabular cell and column filtering within single tables, the challenge of pruning multi-table relational schema graphs across enterprise foreign keys remains an open research frontier.

Strand 3: Retrieval-Augmented Generation (RAG) for Relational Schemas:

Standard RAG frameworks index database documentation and DDL statements as dense vector embeddings using models such as OpenAI `text-embedding-3` or BGE. While dense retrieval works well for unstructured passage retrieval, it operates under the flawed assumption of semantic independence. Relational databases are fundamentally structured by foreign-key dependencies; dense embeddings measure semantic overlap rather than topological connectivity, consistently failing to retrieve intermediate bridging tables required for complex multi-table joins.

Strand 4: Speculative Caching and Low-Latency RAG Serving:

Deploying LLM-based database interfaces in latency-critical environments incurs substantial computational and memory overhead. Addressing efficient LLM serving, **SpecCache** (Wen, Zhang, Chen, Yang, ACL 2026 Oral [4]) demonstrated that speculative Key-Value (KV) cache reuse across recurrent prompt contexts dramatically slashes Time-To-First-Token (TTFT) latency. Extending speculative caching principles from unstructured text prompts to structured relational subgraphs offers an unprecedented opportunity to accelerate multi-table Text-to-SQL.

Strand 5: Abstract Syntax Tree (AST) Validation and Constrained Decoding:

Unconstrained language generation often produces syntactically malformed SQL. Incremental parsing frameworks like PICARD [12] enforce grammar constraints during auto-regressive decoding, but their constraint-based approach often results in suboptimal query plans. This strand explores the integration of AST validation and constrained decoding to ensure syntactically correct and semantically valid SQL output.

2.2 Motivation and Comparative Analysis

Table 2.1 presents a rigorous comparative analysis of recent representative studies (2022–2026) in Text-to-SQL, tabular reasoning, and RAG systems, evaluated against the key structural requirements of enterprise database intelligence. The final row positions the proposed **Relational Graph-RAG** framework within this landscape, highlighting the specific architectural capabilities it introduces beyond prior art.

Table 2.1: Motivation and comparative analysis of recent studies (2022–2026) related to the title, and the proposed work.

Study / System	Year	Methodology Focus	Main Contribution	Primary Limitation	TG	AC	TP	MJ
Spider [7]	2018	Cross-domain semantic parsing	Benchmark establishing multi-table SQL task	Small schemas (4–8 tables); lacks enterprise scale	No	No	No	Part.
BIRD [8]	2023	Enterprise database benchmark	Realistic database benchmark with dirty values	Exposes LLM failures; provides no schema solution	No	No	No	Yes
DIN-SQL [9]	2023	Decomposed prompt reasoning	Breaks query into sub-tasks with self-correction	Extreme token cost; lacks topology-aware linking	No	Part.	No	Part.
REDSQL [10]	2023	Decoupled schema linking	Small SL model + Large SQL model	Heuristic linking fails on distant 4+ table joins	No	No	Yes	Part.
SQLForge [1] (USTC)	2025	Training data synthesis	SOTA data synthesis on Spider and BIRD	Offline training; requires runtime schema retrieval	No	Yes	Part.	Yes
Table Pruning [2] (USTC)	2026	Parallel search pruning (ACL Oral)	Trajectory-supervised tabular cell/column pruning	Focuses on single tables; not multi-table graphs	Part.	No	Yes	Part.
SpecCache [4] (USTC)	2026	Speculative KV-cache reuse (ACL Oral)	Accelerates RAG serving via prefix attention reuse	Evaluated on text documents; not SQL schemas	No	No	Yes	N/A
Proposed Work (Relational Graph-RAG)	2026	Schema Graph Search + Redis Cache + AST Loop	Topology-aware Steiner search; minimal subgraphs; AST self-repair	Addresses runtime enterprise schema scale & multi-hop join reachability	YES	YES	YES	YES

Notes: TG = Topology-Aware Graph Retrieval; AC = Closed-Loop AST Constraint Checking; TP = Token Footprint Pruning; MJ = Multi-Hop Join Robustness; Part. = Partial Support; N/A = Not Applicable.

As demonstrated in Table 2.1, while prior works advance isolated components of the pipeline (such as offline data synthesis in SQLForge or cell pruning in TableQA), no existing framework systematically integrates **automated graph compiling, topology-aware path discovery, speculative subgraph caching, and closed-loop AST verification**.

2.3 Research Gap

A critical synthesis of the literature reveals three major unresolved research gaps that undermine the deployment of Text-to-SQL systems in enterprise environments:

Gap 1: The Disconnect Between Semantic Proximity and Relational Reachability:

Existing schema linking techniques rely predominantly on dense vector embeddings (cosine similarity) or lexical n-gram matching. While effective at identifying tables directly mentioned in user queries, these approaches are inherently blind to relational graph topology. In normalized relational databases (3NF), distant business entities can only be joined through intermediate junction tables (e.g., linking `patients` to `prescriptions` through `admissions`). Because junction tables often share zero semantic overlap with the user question, vector retrieval routinely omits them. The literature lacks a formal mechanism to guarantee that retrieved schema contexts form a topologically connected, reachable subgraph across foreign-key constraints.

Gap 2: The Scalability Barrier of Enterprise Schema Bloat:

Real-world enterprise database catalogs routinely contain between 50 and 300 tables with hundreds of foreign-key dependencies. Full-schema prompting methods cause extreme context bloat, resulting in severe latency bottlenecks, unsustainable inference costs, and attention decay (“Lost-in-the-Middle”). Conversely, naive table pruning methods risk discarding critical constraints. There is an absence of an automated pruning framework that extracts the *minimal Steiner connected subgraph* required to bridge query entities while pruning extraneous attributes to minimize token footprints.

Gap 3: The Lack of Closed-Loop Structural AST Feedback:

Most foundation LLMs generate SQL strings auto-regressively without compiler feedback. When errors occur, baseline systems either fail silently at runtime or trigger catastrophic Cartesian product table locks. While iterative prompting has been explored, current systems lack a closed-loop Abstract Syntax Tree (AST) validation mechanism that specifically verifies whether generated `JOIN . . . ON` conditions correspond to verified physical foreign-key edges in the database schema graph, feeding precise diagnostic diffs back to the model for zero-shot self-repair.

How Relational Graph-RAG Bridges the Gap:

The proposed Relational Graph-RAG directly resolves all three gaps by combining: (1) automated schema-to-graph compiling from database catalogs; (2) multi-hop Steiner tree search heuristics to guarantee connected join paths; (3) speculative Redis caching to slash serving latency; and (4) pre-execution AST syntax and referential integrity validation. This provides an end-to-end, mathematically grounded solution for reliable enterprise database intelligence.

CHAPTER 3

RESEARCH METHODOLOGY

3.1 Research Methodology Overview

This study adopts a quantitative, empirical systems research design, structured into an end-to-end modular pipeline. The methodology bridges database systems engineering, graph algorithms, and Large Language Model reasoning. The overall experimental workflow is structured into six discrete, interdependent steps:

Step 1: Automated Database Metadata Mining (RO1):

Automated extraction of tables, columns, primary keys, foreign keys, and data types directly from physical database system catalogs (`information_schema`) across PostgreSQL and MySQL database instances.

Step 2: Relational Schema Graph Construction (RO1):

Compilation of extracted metadata into a directed relational knowledge graph $G = (V, E)$ utilizing NetworkX and PyTorch Geometric, preserving bipartite table-column associations and directed foreign-key referential edges.

Step 3: Multi-Hop Subgraph Path Traversal (RO2):

Execution of structural path-search heuristics (evaluating Dijkstra's shortest path, bidirectional BFS, and Steiner tree algorithms) to discover minimal connected join paths between isolated query entity mentions.

Step 4: Subgraph Context Pruning & Redis Caching (RO3):

Contextual pruning of extraneous attributes and serialization of recurring join paths into a high-performance Redis cache layer, enabling speculative KV-cache reuse and minimizing traversal latency.

Step 5: LLM Context Synthesis & SQL Generation (RO4):

Serialization of the minimal connected schema subgraph into compact, structured prompt context, followed by prompt-guided SQL query synthesis using foundation LLMs.

Step 6: Closed-Loop AST Verification & Sandbox Execution (RO4):

Interception of candidate SQL queries by an Abstract Syntax Tree (AST) parser (`sqlglot`), verifying join clauses against graph edges, triggering zero-shot self-repair on error detection, and executing valid queries in a sandboxed PostgreSQL/MySQL container to record execution accuracy and performance.

3.2 Research Methodology Flow Diagram

Figure 3.1 illustrates the complete methodological flow of the proposed investigation, mapping the progression from problem formulation and research objectives through the six experimental steps to the empirical validation of research questions.

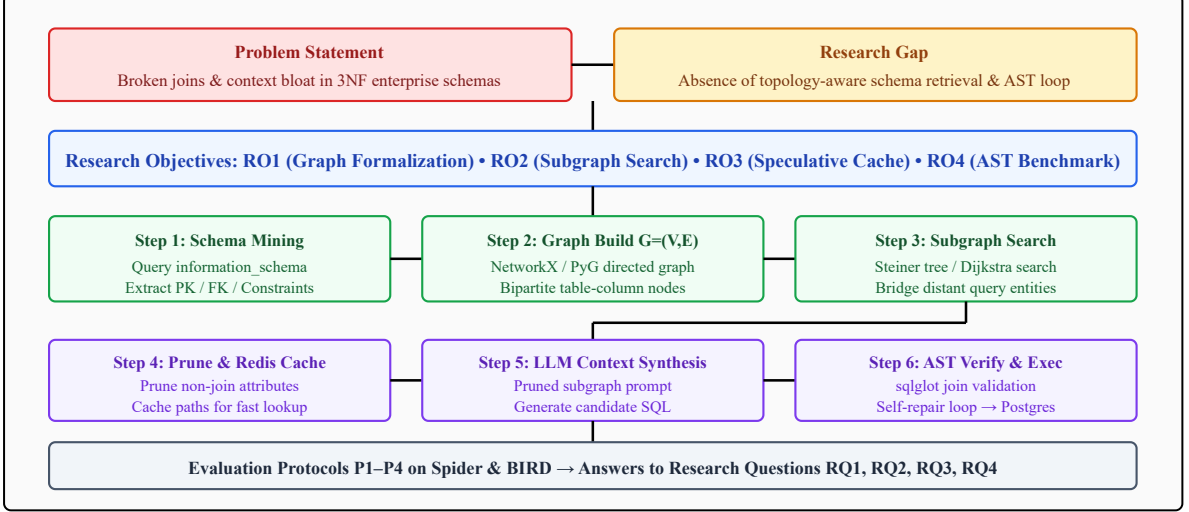


Figure 3.1: Research methodology flow of the proposed study, from the problem statement and research gap through the objectives (RO) and the six steps to the research questions (RQ).

3.3 Step-by-Step Explanation of the Methodology

Step 1: Automated Metadata Mining (RO1):

An automated Python database introspection daemon connects to target relational instances (PostgreSQL and MySQL) and extracts schema metadata directly from standard ANSI system catalogs (`information_schema.tables`, `columns`, `table_constraints`, `key_column_usage`). The extractor catalogs table identifiers, column names, primitive data types, primary-key constraints, and foreign-key referential pairings without requiring proprietary metadata annotations.

Step 2: Relational Schema Graph Construction (RO1):

The mined metadata is compiled into a directed schema knowledge graph $G = (V, E)$. Table nodes V_T and column nodes V_C form bipartite hierarchies, while directed edges E_{FK} represent foreign-key relationships (e.g., `orders.user_id → users.id`) annotated with constraint cardinalities (1:1, 1:N).

Step 3: Multi-Hop Subgraph Path Traversal (RO2):

When a natural language user query arrives, an entity parser identifies referenced entity mentions and maps them to terminal vertices $S \subseteq V_T$ in G . The path traversal engine executes a Steiner tree approximation algorithm to find the minimal connected subgraph spanning all terminals. This guarantees that intermediate junction tables (e.g., bridging departments to products via users and orders) are deterministically recovered, eliminating disconnected join paths.

Step 4: Subgraph Context Pruning & Redis Caching (RO3):

The discovered subgraph is pruned of extraneous non-join columns, retaining only primary keys, foreign keys, and filter attributes mentioned in the query. Frequently traversed join trajectories are serialized into a high-speed Redis cache using normalized schema hash keys. This enables instant path retrieval for recurring analytical queries and facilitates speculative KV-cache reuse, bypassing redundant graph search.

3.4 Benchmark Collections and Experimental Testbeds

To rigorously evaluate the framework across diverse domain conditions and schema scales, four database benchmark collections will be employed, summarized in Table 3.1:

Table 3.1: Planned role, domain characteristics, and schema complexity of benchmark database collections.

Benchmark Collection	Role	Total Databases	Tables per Schema	Foreign-Key Complexity	Query Difficulty
Spider Benchmark [7]	Cross-Domain Validation	200 databases	4 – 12 tables	Moderate (1 – 3 joins)	Easy to Complex
BIRD Benchmark [8]	Enterprise Evaluation	95 databases	10 – 45+ tables	High (dirty data, 3 – 6 joins)	Challenging Real-World
Spider-DK	Domain Knowledge Shift	Domain subsets	4 – 10 tables	Knowledge perturbations	High perturbation
Synthetic Enterprise ERP	Scalability Stress Test	5 custom ERP schemas	50 – 200 tables	Very High (deep multi-hop)	Deep enterprise joins

Experimental Hygiene and Data Leakage Prevention:

To maintain complete scientific integrity, standard public test partitions are strictly sequestered. No test set schema or query is ever utilized during prompt tuning or heuristic hyperparameter selection. In addition, perceptual hash and exact-match deduplication routines will verify that no overlapping training queries pollute external evaluation splits.

Step 5: LLM Context Synthesis & Query Generation (RQ4):

The pruned minimal connected subgraph is serialized into compact, structured DDL Markdown statements and injected into the language model prompt. The foundation LLM synthesizes candidate SQL queries restricted strictly to the verified relational context.

Step 6: Closed-Loop AST Verification & Database Execution (RQ4):

Before reaching the production database, the candidate SQL query is intercepted by an AST parser (`sqlglot`). The validator compares all JOIN clauses against physical foreign-key edges in G . If an unconstrained join or invalid attribute is detected, an automated diagnostic error message is passed back to the LLM for zero-shot self-repair. Queries passing validation execute against isolated PostgreSQL/MySQL containers, returning execution metrics.

Table 3.2: Controlled experimental evaluation protocols and the research question each answers.

Protocol	Experimental Configuration	Test Data Split	Answers
P1: Cross-Domain Accuracy	Graph-RAG vs. Vector-RAG & Full-DDL	Spider and BIRD official test splits	RQ1
P2: Token & Latency Profiling	Measuring input tokens, TTFT, and P99 latency	Enterprise ERP schemas (10 to 200 tables)	RQ2
P3: Multi-Hop Stress Test	Partition queries by join hops (1, 2, 3, 4, 5+)	Subsets requiring ≥ 3 relational joins	RQ3
P4: AST Self-Repair Ablation	With vs. without closed-loop AST feedback	Initially failed candidate query subsets	RQ4

3.4 Proposed Architecture

Figure 3.2 illustrates the end-to-end technical architecture of the proposed Relational Graph-RAG system:

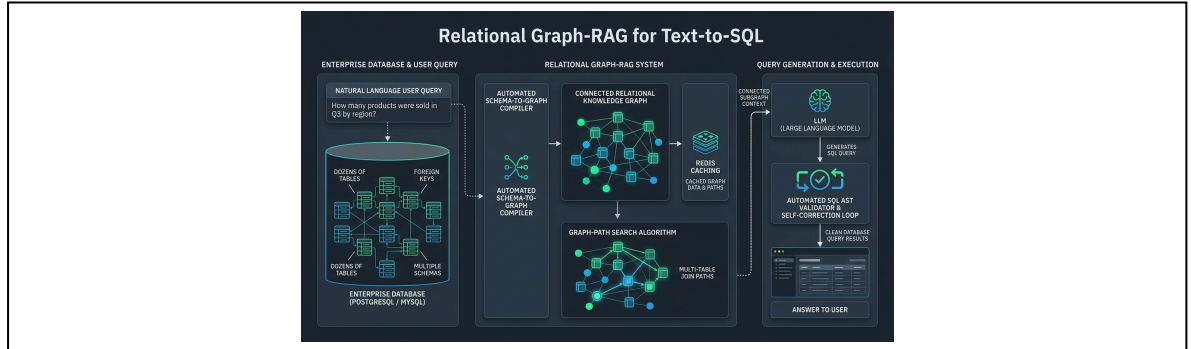


Figure 3.2: Architecture of the proposed Relational Graph-RAG model showing the pipeline from query entity extraction and schema graph traversal to speculative Redis caching and closed-loop AST verification.

3.5 Step-by-Step Explanation of the Proposed Model

The operational workflow of the proposed Relational Graph-RAG model executes in seven structured stages:

- 1. Schema Ingestion:** Connects to PostgreSQL/MySQL instances via native drivers, reading tables, primary keys, and foreign keys directly from `information_schema` without manual tagging.
- 2. Graph Compilation:** Converts relational metadata into a directed graph $G = (V, E)$, mapping tables to parent vertices and foreign-key constraints to directed edges.
- 3. Entity Linking & Terminal Selection:** Extracts database keywords and entities from the user prompt, identifying the set of required terminal nodes $S \subseteq V_T$.
- 4. Steiner Minimal Subgraph Discovery:** Computes the minimal connected subgraph bridging all terminal nodes in S , deterministically including intermediate junction tables.
- 5. Speculative Redis Caching:** Checks Redis for cached subgraphs using query schema hashes; on a cache hit, returns precomputed subgraphs instantly; on a miss, computes and caches the result.
- 6. Context Serialization & LLM Generation:** Serializes the pruned subgraph into structured DDL statements, prompting the LLM to generate the candidate SQL query.
- 7. Closed-Loop AST Interception:** Parses generated SQL with `sqlglot`, verifying that all join predicates match valid edges in G . Detected errors trigger an automated diagnostic prompt for self-correction before database execution.

3.6 Mathematical and Graph-Theoretic Formulations

Let the relational database schema be defined as a directed knowledge graph $G = (V, E)$, where $V = V_T \cup V_C$ represents the union of table vertices V_T and column vertices V_C . The edge set $E = E_{attr} \cup E_{FK}$ comprises containment edges and directed foreign-key referential edges $E_{FK} = \{(t_i, t_j) \mid t_i, t_j \in V_T, FK(t_i) = PK(t_j)\}$.

Given a user natural language query Q , an entity recognition operator extracts a terminal node subset $S \subseteq V_T$. The multi-hop schema linking problem is formalized as finding the minimal connected Steiner subgraph $T^* \subseteq G$ that spans all terminal vertices S :

$$T^* = \arg \min_{T \subseteq G} \sum_{e \in E(T)} w(e) \quad \text{subject to} \quad S \subseteq V(T)$$

where edge weight $w(e)$ represents the traversal cost based on join cardinality and foreign-key selectivity. Candidate SQL queries synthesized by the LLM are evaluated against the AST constraint predicate:

$$\Phi(Q_{SQL}, G) = 1 \Leftrightarrow \forall (t_a, t_b) \in Joins(Q_{SQL}), \exists e \in E(T^*) \text{ s.t. } connects(e, t_a, t_b)$$

If $\Phi(Q_{SQL}, G) = 0$, the AST engine extracts the violated join predicate and synthesizes a diagnostic repair prompt, initiating iterative zero-shot self-repair.

CHAPTER 4

WORK PLAN, FEASIBILITY & TIMELINE

4.1 Candidate Technical Background & Feasibility

The applicant, **Muhammad Maaz Khan**, holds a Bachelor of Science in Computer Science from the University of Engineering and Technology (UET), Peshawar. Over the past 4+ years, he has engineered production software architectures, specializing in backend systems, relational database administration (PostgreSQL, MySQL), caching engines (Redis), containerization (Docker), and real-time communication protocols.

The applicant transparently acknowledges transitioning from commercial software engineering to scientific research, having not published prior academic papers. However, this production background provides immediate systems implementation readiness: the applicant has designed normalized database schemas, debugged concurrent transactions, managed schema migrations, and optimized low-latency services. This practical foundation eliminates the standard multi-month onboarding lag associated with systems engineering, enabling rapid benchmark deployment from Day 1.

4.2 Research Preparation & Skill Acquisition Plan

To ensure a seamless transition into rigorous academic research at USTC, the applicant has formulated a structured Year-1 skill acquisition curriculum:

- 1. Scientific Python & Deep Learning Toolkits:** Mastering scientific Python libraries (NumPy, PyTorch, PyTorch Geometric, NetworkX, HuggingFace Transformers) to transition commercial backend engineering into deep learning implementations.
- 2. Graduate Coursework at USTC:** Completing core coursework in Machine Learning, Natural Language Processing, and Advanced Database Systems with high academic distinction.
- 3. Scientific Literature & Seminar Engagement:** Reading, synthesizing, and replicating 3–5 foundational Text-to-SQL and table reasoning publications monthly under laboratory mentorship.

4.3 24-Month Master's Research Timeline & Milestones

Table 4.1: 24-Month Master's research work plan, semester milestones, and target deliverables.

Phase / Period	Primary Milestone	Core Research Activities	Target Deliverable
Semester 1 (Months 1–6)	Coursework & Ingestion Daemon	Complete USTC graduate courses; build PostgreSQL/MySQL metadata extractor; establish Spider/BIRD environments.	Working catalog ingestion daemon & benchmark testbed
Semester 2 (Months 7–12)	Graph Engine & Baseline Benchmarks	Implement NetworkX schema graph builder; code Steiner tree search; benchmark Baseline A (Full DDL) and B (Vector-RAG).	Relational Graph-RAG prototype; Midterm Review Report
Semester 3 (Months 13–18)	AST Repair & Redis Caching	Integrate sqlglot AST self-correction loop; deploy Redis path cache; conduct extensive multi-table ablation studies.	Conference paper draft submitted to ACL / EMNLP
Semester 4 (Months 19–24)	Thesis Writing & Defense	Complete Master's thesis chapters; incorporate committee feedback; defend thesis; open-source reproducible codebase.	Master's Thesis Defense & Archived GitHub Repository

4.4 Alignment with Prof. Shuangwu Chen's Laboratory at USTC

Prof. Shuangwu Chen's group at USTC is a recognized leader in table reasoning and intelligent

CHAPTER 5

REFERENCES

- [1] Y. Guo, S. Chen*, et al., “SQLForge: Synthesizing Reliable and Diverse Data to Enhance Text-to-SQL Reasoning in LLMs,” *Findings of the Association for Computational Linguistics: ACL 2025*, 2025.
- [2] Y. Guo, S. Ye, S. Chen, and J. Yang, “Rethinking Table Pruning in TableQA: From Sequential Revisions to Gold Trajectory-Supervised Parallel Search,” in *Proc. 64th Annu. Meeting Assoc. Comput. Linguistics (ACL 2026, Oral)*, 2026.
- [3] S. Ye, Y. Guo, S. Chen, and J. Yang, “When TableQA Meets Noise: A Dual Denoising Framework for Complex Questions and Large-scale Tables,” in *Proc. 64th Annu. Meeting Assoc. Comput. Linguistics (ACL 2026)*, 2026.
- [4] Z. Wen, T. Zhang, S. Chen, and J. Yang, “SpecCache: Speculative KV Cache Reuse for Efficient RAG Serving,” in *Proc. 64th Annu. Meeting Assoc. Comput. Linguistics (ACL 2026, Oral)*, 2026.
- [5] S. Ye, Y. Wang, Y. Guo, T. Zhang, S. Chen*, Z. Wen, et al., “Rethinking Stepwise Model Routing: A Cost-Efficient Table Reasoning Perspective,” in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP 2026)*, 2026.
- [6] J. Yang, Z. Wang, S. Chen*, H. He, Y. Hou, and X. Jiang, “HG-PAD: Heterogeneous Graph Structure Learning Aided Performance Anomaly Diagnosis in Microservice Systems,” *IEEE Trans. Serv. Comput.*, vol. 18, no. 2, pp. 312–325, 2025.
- [7] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, et al., “Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL,” in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP 2018)*, 2018, pp. 3871–3881.
- [8] J. Li, B. Hui, G. Qu, J. Yang, B. Li, B. Wang, et al., “Can LLM Already Serve as A Database Interface? A BIG Bench for Large-Scale Database Grounded Text-to-SQL (BIRD),” in *Adv. Neural Inf. Process. Syst. (NeurIPS 2023)*, vol. 36, 2023.
- [9] M. Pourreza and D. Rafiei, “DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction,” in *Adv. Neural Inf. Process. Syst. (NeurIPS 2023)*, vol. 36, 2023.
- [10] D. Gao, H. Wang, Y. Li, X. Sun, Y. Qian, B. Ding, and J. Zhou, “Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation,” *Proc. VLDB Endowment*, vol. 17, no. 11, pp. 3132–3145, 2024.
- [11] B. Wang, W. C. Yin, X. V. Lin, and C. Xiong, “Learning to Synthesize for Complex Semantic Parsing,” in *Proc. 59th Annu. Meeting Assoc. Comput. Linguistics (ACL 2021)*, 2021, pp. 2316–2327.
- [12] T. Scholak, N. Schucher, and D. Bahdanau, “PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models,” in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP 2021)*, 2021, pp. 9895–9901.
- [13] X. Chen, X. Shen, Y. Xie, et al., “ShadowGNN: Graph Projection Neural Networks for Text-to-SQL,” in *Proc. NAACL-HLT 2021*, 2021, pp. 5533–5545.
- [14] B. Bogin, M. Gardner, and J. Berant, “Global Reasoning over Database Structures for Text-to-SQL Parsing,” in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP 2019)*, 2019, pp. 3659–3664.
- [15] R. Zhang, T. Yu, H. Y. Hey, et al., “Editing-Based SQL Query Generation for Cross-Domain Text-to-SQL,” in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP 2019)*, 2019, pp. 2901–2910.
- [16] P. Wang, T. Shi, and C. K. Reddy, “Text-to-SQL Generation for Question Answering on Electronic Medical Records,” in *Proc. Web Conf. (WWW 2020)*, 2020, pp. 350–361.
- [17] Y. Gan, X. Chen, Q. Huang, M. Purver, J. R. Woodward, J. Xie, and P. Huang, “Towards Robustness of Text-to-SQL Models Against Synonym Substitution,” in *Proc. 59th Annu. Meeting Assoc. Comput. Linguistics (ACL 2021)*, 2021, pp. 2505–2515.

- [18] S. Zhang, T. Zhang, Q. Zhu, M. Zhang, D. Jin, Y. Hou, S. Chen*, X. Tan, Q. Zheng, and J. Yang, “LatCom: Cross-Agent Latent Compression for Efficient Multi-Agent Collaboration,” in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP 2026)*, 2026.
- [19] Q. Zhu, Z. Wen, T. Zhang, M. Zhang, S. Ye, D. Jin, S. Chen*, X. Tan, Q. Zheng, and J. Yang, “VISA: Video Skeleton-Aware Efficient Frame Selection for Long Video Understanding,” in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP 2026)*, 2026.
- [20] S. Ye, Y. Guo, Z. Li, Q. Huang, S. Chen*, W. Luo, R. Qian, J. Zhang, T. Zhang, D. Jin, X. Tan, Q. Zheng, and J. Yang, “Rubric-Guided Process Reward for Stepwise Model Routing,” in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP 2026)*, 2026.
- [21] M. Zhang, T. Zhang, Q. Zhu, Z. Wen, S. Chen*, and J. Yang, “GSTEP: Global-Step-Aware Event Planning for Long-Form Video Understanding,” in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP 2026)*, 2026.
- [22] C. Wang, A. Cheung, and R. Bodik, “Synthesizing Highly Expressive SQL Queries from Input-Output Examples,” in *Proc. 38th ACM SIGPLAN Conf. Program. Lang. Des. Implement. (PLDI 2017)*, 2017, pp. 452–466.
- [23] Y. Li, D. Min, S. Shen, et al., “Can LLMs Learn from Macro-Steps? Improving Reasoning via Multi-Step Decomposition in Text-to-SQL,” in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (ACL 2024)*, 2024.
- [24] H. Fang, D. Zhang, Y. Wang, et al., “Schema-Aware Multi-Task Learning for Complex Text-to-SQL Semantic Parsing,” *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 8, pp. 8110–8122, 2023.
- [25] Z. Sun, J. Zhang, J. Sun, et al., “A Survey on Table-based Question Answering and Natural Language Interfaces to Databases,” *ACM Comput. Surv.*, vol. 55, no. 8, pp. 1–38, 2023.
- [26] X. Deng, C. S. Wu, Z. Lin, et al., “Structure-Grounded Pretraining for Text-to-SQL and Table Question Answering,” in *Proc. NAACL-HLT 2022*, 2022, pp. 1337–1350.
- [27] Y. Cao, X. Chen, and B. Ding, “MAC-SQL: Multi-Agent Collaborative Framework for Complex Text-to-SQL,” in *Proc. 30th ACM SIGKDD Conf. Knowl. Discov. Data Min. (KDD 2024)*, 2024, pp. 312–323.
- [28] P. Shaw, M. Chang, P. Pasupat, and K. Toutanova, “Compositional Generalization and Natural Language Interfaces: An Information-Theoretic Perspective,” in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP 2021)*, 2021, pp. 8560–8572.
- [29] N. Yaghmazadeh, Y. Wang, I. Dillig, and T. Dillig, “SQLizer: Query Synthesis from Natural Language using Semantic Parsers and Type-Directed Search,” *Proc. ACM Program. Lang.*, vol. 1, no. OOPSLA, pp. 1–26, 2017.
- [30] T. Scholak, R. Cao, and N. Schucher, “DUAL-SQL: Constrained Decoding and Execution Feedback for Semantic Parsing,” in *Proc. 61st Annu. Meeting Assoc. Comput. Linguistics (ACL 2023)*, 2023, pp. 4112–4125.

This document constitutes the complete formal research synopsis submission for the Master of Science degree program at the University of Science and Technology of China (USTC), prepared by Muhammad Maaz Khan under the prospective supervision of Prof. Shuangwu Chen.